CSE310: Compiler Sessional, January 2026 Term

ICG Online Assignment (B2)

Implementation of Compound Assignments

Time: 40 Mins Total Marks: 10

Problem

In this online, you are required to extend the grammar to support **compound assignment operators +=, -=, *=, /= and %=**. These operators combine an arithmetic operation with assignment. The arithmetic operation is performed between the left and right operands and then the result is assigned to the left operand. The right operand can be any expression. Thus, your program should support the following syntax.

```
int a, b;
a += -b + 5;
a -= b++;
a *= b % 3;
a /= 2 * 3 + 5;
a %= b;
```

Task

1. Extend the current grammar to support **compound assignment operators +=, -=, *=, /= and %=**. Note that these operators have the same precedence as the assignment operator (=). Therefore, the arithmetic operation associated with an compound assignment is performed after the expression on the right is fully evaluated. Like the offline, you need to only consider integer type variables and integer operations. There will be no syntax or semantic errors in the given input program.
2. Update your ICG code to generate assembly code for this new rule/syntax. The assembly code output by your program must correctly be assembled with 'fasm' . Subsequently, the binary produced by 'fasm' for against your assembly code must produce the expected output in the input program.

Mark Distribution

- Extending the grammar to support compound assignment operators. (3)
- Code generation for compound assignment operators. (7)

Find a sample input-output in the next pages.

Sample Input-Output

Input Program

```
1 int a;
2 int main()
3 {
4     int b;
5     a=10;
6     b=20;
7     println(a);
8     println(b);
9     a += b + 5;
10    println(a);
11    a -= 2 * -b;
12    println(a);
13    a *= b;
14    println(a);
15    a /= 11;
16    println(a);
17    a %= 11;
18    println(a);
19    return 0;
20 }
```

Expected Output (from the generated binary by fasm)

```
10
20
35
75
1500
136
4
```

Generated Sample Assembly (print library omitted)

```
format ELF executable 3
entry main
segment readable writeable
      a dd 1 DUP (0)
segment readable executable
main:
      PUSH EBP
      MOV EBP, ESP
      SUB ESP, 4
.L1:
      MOV EAX, 10        ; Line 5
      MOV [a], EAX
      PUSH EAX
      POP EAX
.L2:
      MOV EAX, 20        ; Line 6
      MOV [EBP-4], EAX
      PUSH EAX
      POP EAX
.L3:
      MOV EAX, [a]       ; Line 7
      CALL print_number
.L4:
      MOV EAX, [EBP-4]       ; Line 8
      CALL print_number
.L5:
      MOV EAX, 5       ; Line 9
      MOV EDX, EAX
      MOV EAX, [EBP-4]       ; Line 9
      ADD EAX, EDX
      PUSH EAX
      POP EAX       ; Line 9
      MOV EDX, EAX
      MOV EAX, [a]       ; Line 9
      ADD EAX, EDX
      MOV [a], EAX
      PUSH EAX
      POP EAX
.L6:
      MOV EAX, [a]       ; Line 10
      CALL print_number
.L7:
      MOV EAX, [EBP-4]       ; Line 11
      NEG EAX
      PUSH EAX
      POP EAX       ; Line 11
      MOV ECX, EAX
      MOV EAX, 2       ; Line 11
      CWD
      IMUL ECX
      PUSH EAX
      POP EAX       ; Line 11
      MOV EDX, EAX
      MOV EAX, [a]       ; Line 11
      SUB EAX, EDX
      MOV [a], EAX
      PUSH EAX
```

```
        POP EAX
.L8:
        MOV EAX, [a]        ; Line 12
        CALL print_number
.L9:
        MOV EAX, [EBP-4]        ; Line 13
        MOV ECX, EAX
        MOV EAX, [a]        ; Line 13
        CWD
        IMUL ECX
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L10:
        MOV EAX, [a]        ; Line 14
        CALL print_number
.L11:
        MOV EAX, 11        ; Line 15
        MOV ECX, EAX
        MOV EAX, [a]        ; Line 15
        CWD
        IDIV ECX
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L12:
        MOV EAX, [a]        ; Line 16
        CALL print_number
.L13:
        MOV EAX, 11        ; Line 17
        MOV ECX, EAX
        MOV EAX, [a]        ; Line 17
        CWD
        IDIV ECX
        MOV EAX, EDX
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L14:
        MOV EAX, [a]        ; Line 18
        CALL print_number
.L15:
        MOV EAX, 0        ; Line 19
        JMP .L17
.L16:
.L17:
        ADD ESP, 4
        POP EBP
        MOV EAX,1
        XOR EBX, EBX
        INT 0x80
        POP EBP
        RET
```